

Deepfake and Manipulated Media Detection

A triage service for video, image and audio — auditable, and explicit about what it does not know.

SCOPE	DEPLOYMENT	INTERFACE	STATUS
MVP, timeline reopened	Docker Compose, single GPU node	Web application	Pipeline verified end to end

1 Problem Understanding

Fake and manipulated media — face swaps, puppeted video, cloned voices — is now cheap to produce and spreads quickly. On social platforms, most of the damage happens in the first few hours, before any clip can be checked by hand. The goal is a system that helps catch this content early. Four constraints shape the design:

- The output is a signal, not a verdict.** No detector is right all the time, and accuracy drops sharply on media the model was never trained on. A confident "FAKE" label is more dangerous than useful — a false positive on real footage can wrongly implicate someone. The system reports a score, a band, and an explicit "not sure" range, not a binary label.
- Every upload is hostile.** Media decoders have a long history of memory-safety bugs, and detection scores can be manipulated by an attacker who perturbs the input in small ways. Both risks are handled architecturally rather than assumed away.
- The media is biometric.** What gets uploaded is faces and voices. Retaining it indefinitely increases breach exposure and legal risk, so deletion is the default and must be provable — while keeping enough evidence that a disputed result can still be reviewed.
- A result has to be reconstructible.** If a decision is challenged later, a raw score isn't enough. The system records which model version ran, how per-item scores were combined, the settings used, and a hash of the analyzed bytes — captured at decision time, not after.
- Scope.** The system looks for signs that media was synthesized or edited, and routes the result. It does not determine ground truth and does not make takedown decisions — those are separate, downstream questions.

2 Proposed Solution

The service accepts a video, image, or audio file and returns three things: a calibrated manipulation-likelihood score, a band that determines what happens next, and an audit record that makes the result reproducible. Uploaded media is deleted on a timer by default.

Unified decision point

Each media type runs through a pipeline suited to it, but all three converge on the same aggregation, band routing, and retention logic — one place where a score becomes a decision, instead of three that can drift apart independently.

Careful score combination

A model scores individual frames, faces, or audio chunks. Scores are combined with a confidence-weighted trimmed mean rather than a plain average, so a handful of badly cropped faces can't skew the result; items are weighted down by how confident the face detection was.

No silent guessing

If face detection finds nothing, the result is marked *undetermined* — never defaulted to "authentic." If more than one face is present, each is scored separately and the frame takes the worst score, so a single edited face in a crowd isn't averaged away.

Score bands

SCORE	BAND	WHAT HAPPENS
Under 20	Likely authentic	Cleared automatically, deleted on schedule
20–40	Leaning authentic	Cleared automatically, deleted on schedule
40–60	Uncertain	Flagged for review, alert sent
60–80	Leaning manipulated	Flagged, low-urgency alert; still deleted on schedule
Over 80	Likely manipulated	Flagged and alerted; extended retention window opens
None	Undetermined	No usable signal — flagged, never scored as real or fake

The 60–80 band is handled conservatively: calibration near the top threshold isn't trusted enough to surface a raw percentage, so results here are logged and flagged rather than passed straight through.

3 System Architecture

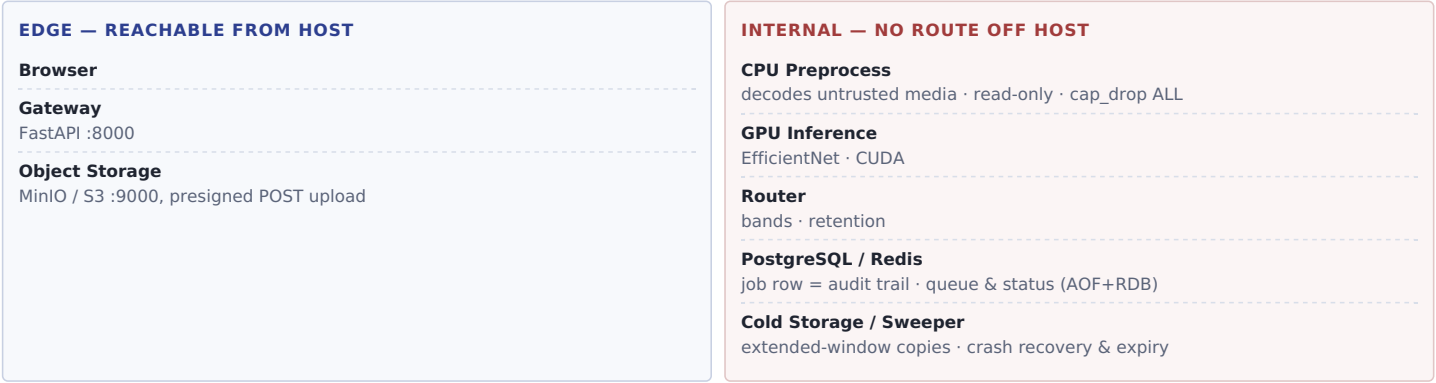
Detection pipelines



The face model is shared between the video and image pipelines (one model_version_id); audio uses a separate model with its own. For images, aggregation is the identity function — there's a single item to score.

Trust boundary

The network layout is a security control: the container that opens attacker-supplied media has no route off the machine at all.



No AV scanning is deployed. Container isolation and lockdown substitute for it, and shipped alongside the worker that parses untrusted input — not after it.

Request flow

1. The client requests a job. The gateway rate-limits the request, creates the database row, and returns a signed upload grant.
2. The client uploads directly to object storage — large media never passes through the API.
3. The client notifies the gateway; the job is queued. Repeat calls are idempotent and won't double-queue it.
4. The CPU worker decodes the file, samples frames, and detects and aligns faces (or chunks and spectrograms audio).
5. The GPU worker scores every item and writes a row per item.
6. The router combines scores, picks a band, raises any flags, and starts deletion of the working media.
7. Status updates stream over a WebSocket throughout, with polling as a fallback if the connection drops.

4 Key Features

Deletion you can verify TIER 1

Uploads and derived face crops are deleted once inference finishes. Every deletion path — the normal delete, the crash-recovery sweep, and the cold-storage timer — checks a hold flag first. The flag column and its check were written in the same commit as the delete code, before anything could set the flag.

The job row is the audit trail TIER 1

Every result stores a SHA-256 hash of the analyzed bytes, the ID of the model that produced the scores, and the exact combination method and settings. Per-item scores are kept after media deletion since they're numeric rather than biometric — a disputed result can be examined without keeping anyone's face on disk.

An upload path that can't be used as free storage TIER 1

Uploads go straight to storage, so the API never sees those bytes and can't check size directly. The upload grant instead carries a signed size limit enforced by object storage itself — a plain signed URL can't express a size limit at all.

Extended retention window TIER 2

Results in the highest band get their driving face crops — not the full original file — copied to cold storage on a fixed 30-day timer, so a dispute has specific evidence to examine while the upload itself is still deleted on schedule.

This is not a legal hold. It's a fixed timer that expires on its own, described that way in the code, the API responses, and this document. Relying on it for an actual dispute needs legal sign-off, which this codebase does not claim to have.

Isolation instead of virus scanning TIER 3 · SUBSTITUTE

The worker that opens uploaded media has no antivirus in front of it. Instead it runs on an internal-only network with no route off the machine, a read-only filesystem, every Linux capability dropped, no privilege escalation, a non-executable temp directory, and hard limits on memory and process count.

Failing in ways you can see

Dead-letter queue — a job retries a limited number of times, then stops with a recorded reason; never silently lost, never retried forever. **Durable status** — Redis runs with persistence enabled and the service refuses to start without it, so a restart can't wipe in-flight job status. **Recovery sweeps** — background sweeps find finished, failed, or abandoned jobs and clean up any media they left behind, respecting the hold flag.

Every result states how far it can be trusted

Results carry two warnings: one that the score can be adversarially manipulated, another indicating how trustworthy the model itself is. The second fails safe by design — if the system can't determine which model produced a score, it returns the strongest warning rather than staying quiet.

5 Technology Stack

LAYER	CHOICE	WHY
API	Python 3.11, FastAPI, Uvicorn	Async support for pushing live status over WebSockets; typed request/response models
Models	PyTorch, EfficientNet	Good accuracy for the compute cost; separate models for faces and audio, each calibrated independently
Calibration	Temperature scaling	Cheap, needs only a held-out set; a launch-time snapshot, not continuously updated
Database	PostgreSQL 16	Transactional audit trail; JSON columns for aggregation settings; partial indexes keep cleanup sweeps fast
Queue & status	Redis 7, consumer groups	One dependency covers both the work queue and live status; a dead worker's job can be picked up without waiting for a restart
Object storage	MinIO (local), S3 (deployed)	Identical upload behavior in both — MinIO simply drops out of the setup at deploy time
Packaging	Docker, Docker Compose	One image per service; network isolation is written down, not assumed
GPU	NVIDIA CUDA via WSL2	Verified on an RTX 4060 (8GB) before any model code was wired in
Testing	pytest, plus live checks	154 automated tests, plus scripts checking the running system against a real database, Redis, and bucket
Supply chain	gitleaks	Full-history secrets scan runs before anything is pushed publicly

Why not Kubernetes yet: a cluster, secrets, ingress, and storage volumes are days of setup that buy nothing over Compose on a single GPU machine. Compose runs the same services, so moving later is a packaging change, not a redesign. The trigger to move would be a second machine, a real uptime requirement, or a need to scale GPU workers — not simply having time available.

6 Implementation Plan

Harder work is deferred in named tiers so nothing is skipped quietly. The phases below are what was actually built, in order; the original five-day deadline has since been lifted, so they now describe dependency order rather than a fixed schedule.

PHASE	DELIVERED	CONTENTS
Day 1	Skeleton and contracts	Database schema (incl. hold flag), config, storage layer, queue and retry behavior, rate limiting
Day 2	Ingest and isolation	Signed uploads, API endpoints, live status; CPU worker shipped together with its container lockdown, not after it
Day 3	Scoring and routing	Inference workers behind a swappable backend, score combining, worst-case rollup for multiple faces, band routing, review flags
Day 4	Retention	Timed deletion, hold-flag check on every delete path, extended retention window, recovery sweeps
Day 5	Verification	Full stack running together, checks against a real database and real storage, GPU access verified

Current status

- ✓ Full stack runs end to end; every live check passes
- ✓ 154 automated tests passing, plus ~60 checks against the running system
- ✓ Deletion confirmed by inspecting the bucket directly, not by trusting a database flag
- ✓ Hold flag proven to block deletion on every path that can delete
- ✓ Audit trail verified by reading database columns back, not by trusting the API response
- ✓ Upload size limit enforced by storage itself, confirmed by attempting to exceed it
- ✓ Database enforces one score row per item
- ✓ Each result is tied to the model that produced it; a job is refused rather than guessed at if two models are involved
- ✓ Model-trust warning fails safe by design
- Trained weights not yet loaded — a placeholder scorer is active by default

What's next

- Load real face and audio weights, then run calibration separately for each model
- Validate score bands against real score distributions — currently based on placeholder output
- Replace the review flag with a usable reviewer interface once a reviewer exists
- Build the adversarial-input classifier — the largest known gap, deliberately not partially built

Limitations

- Not robust against a deliberate attacker who edits input to shift the score
- No legal hold — the extended retention window is a self-expiring timer
- Calibration is a launch-time snapshot; it is not production-validated or kept current
- No compliance claim is made — scoped deletion and a partial audit trail help, but meeting any specific regulation is a legal determination this codebase doesn't make
- Detection accuracy is unproven: the default backend derives a score from a hash of the input as a placeholder, carries no detection meaning, and states this on every result

What has been verified is that the pipeline works end to end — ingest, scoring, combination, routing, deletion, and the audit trail. Detection accuracy has not been verified, because it depends on weights that are not loaded yet.